Programming Language Compiler

CMPS403 —Compilers

Team:
Fatma Gamal 1210275
Aisha Tawfik 1210247
Mohab Yasser 1210312

1. Project Overview

This project implements a compiler for a simple, custom programming language. The compiler is built using Flex for lexical analysis and Bison for parsing, and it translates source code into an intermediate representation called quadruples. It also performs semantic analysis and maintains a scoped symbol table throughout the compilation process.

The language we designed is C-like in structure but with some differences. It supports five primitive types (bool, int, float, char, string), constants, functions with typed parameters and return values, and a full set of control flow constructs including if-else, while, repeat-until, for loops with a custom from/to/step syntax, and switch-case with an optional default branch.

When the compiler is given a source file, it produces four output files: a quadruples file containing the intermediate representation, a symbol table log, a visual symbol table showing each scope, and a semantic analysis report containing any semantic errors found during compilation.

2. Tools and Technologies

- Flex (Lexical Analyzer Generator): Used to implement the lexical analyzer. Token rules are written using regular expressions in a .l file, and Flex generates the corresponding C tokenizer code automatically.
- Bison (Parser Generator): Used to implement the parser. The grammar is defined in a .y file using BNF-style production rules, and Bison generates an parser in C. Semantic actions and quadruple generation are embedded directly inside the grammar rules.
- C (C11): The implementation language for all supporting components, including the symbol table data structures, helper utilities, and global compiler state.
- GCC: The C compiler used to compile all generated and hand-written .c files into the final executable.
- Make: A build automation tool. Running make triggers Bison, then Flex, then GCC in the correct order and links everything into the final Compiler.exe.

3. Language Design

The language is designed to be simple and readable while covering all the essential constructs of a structured programming language. Below is an overview of the supported features.

3.1 Types and Declarations

- Primitive types: bool, int, float, char, string
- Variable declaration without initialization: int x;
- Variable declaration with initialization: int x = 5;
- Constant declaration: const int MAX = 100; — constants cannot be reassigned after declaration.
- void is used exclusively as a function return type and cannot be used for variables.

3.2 Expressions

- Arithmetic: +, -, *, /, % (integers only), ^ (exponentiation)
- Logical: & (AND), | (OR), ! (NOT) — operands must be of type bool
- Comparison: ==, !=, <, <=, >, >= — always produces a bool result
- Unary negation: -expr for numeric types
- Operator precedence (highest to lowest): parentheses > unary > ^ > * / % > + - > comparisons > logical

3.3 Control Flow

- Conditional: if (cond) { } and if (cond) { } else { }
- While loop: while (cond) { }
- Repeat-until loop: repeat { } until (cond); — body always executes at least once
- For loop: for x from (start) to (end) { } with optional step (n)
- Switch: switch (expr) { case (val) { } default { } }

3.4 Functions

- Declared at global scope with a return type (or void) and a parameter list
- Called with an argument list; argument types are checked against parameter types
- return expr; is required for non-void functions
- Functions cannot be declared inside other functions

3.5 Scoping

- Every { } block introduces a new scope
- Variables declared in inner scopes shadow variables with the same name in outer scopes
- The symbol table is stack-based: a new scope table is pushed on { and popped on }

3.6 Built-in: print

- print(expr1, expr2, ...) prints one or more values
- print() with no arguments prints a newline

4. Token List

The following table lists all tokens recognized by the lexical analyzer. Comments (// ... and /* ... */) are consumed by the lexer and are not passed to the parser as tokens

.

Token	Pattern / Example	Description
INTEGER	42, 0, 100	A whole number literal. Stored as int.
FLOAT	3.14, 0.5	A floating-point number literal. Stored as float.
BOOL	true, false	A boolean literal. Internally stored as int (1 or 0).
CHAR	'a', '\n', '\t'	A single character literal. Supports escape sequences (\n \t \r \0 \\ \' \").
STRING	"hello\nworld"	A string literal in double quotes. Supports the same escape sequences as CHAR.
TYPE	int, float, bool, char, string	Type specifier used in variable/constant declarations and function parameters.
VOID	void	Return type for functions that do not return a value.
CONST	const	Marks a declaration as a constant. The value cannot be changed after declaration.
IF	if	Starts a conditional statement.
ELSE	else	Optional alternative branch of an if statement.
SWITCH	switch	Starts a switch statement.
CASE	case	A branch inside a switch statement.
DEFAULT	default	The fallback branch inside a switch statement.
FOR	for	Starts a for loop.
FROM	from	Specifies the starting value of the for loop iterator.
TO	to	Specifies the ending value of the for loop iterator.
STEP	step	Optional. Specifies the increment per iteration in a for loop.
WHILE	while	Starts a while loop.
REPEAT	repeat	Starts a repeat-until loop. The body always executes at least once.
UNTIL	until	Specifies the termination condition of a repeat-until loop.
RETURN	return	Returns a value from a function.
PRINT	print	Built-in output statement.
ASSIGN	=	Assignment operator.
PLUS	+	Addition operator.
MINUS	-	Subtraction operator or unary negation.
MULT	*	Multiplication operator.

DIV	/	Division operator. Division by zero is caught as a semantic error.
MOD	%	Remainder operator. Integer operands only.
POW	^	Exponentiation operator.
AND	&	Logical AND. Both operands must be boolean.
OR	|	Logical OR. Both operands must be boolean.
NOT	!	Logical NOT. Operand must be boolean.
LT	<	Less than comparison.
LE	<=	Less than or equal to comparison.
GT	>	Greater than comparison.
GE	>=	Greater than or equal to comparison.
EQ	==	Equal to comparison.
NE	!=	Not equal to comparison.
SEMICOLON	;	Statement terminator.
COMMA	,	Separator in parameter and argument lists.
OPENING_PARENTHESIS	(	Opens a grouped expression or argument list.
CLOSING_PARENTHESIS	)	Closes a grouped expression or argument list.
SCOPE_START	{	Opens a new block and pushes a new scope onto the symbol table.
SCOPE_END	}	Closes a block and pops the current scope from the symbol table.
IDENTIFIER	x, myVar, foo123	A user-defined name for a variable, constant, or function. Must start with a letter or underscore.

5. Symbol Table Design

The symbol table is implemented as a stack of scopes. Each time a { is encountered, a new scope is pushed onto the stack. Each time } is reached, the current scope is popped and destroyed. Lookup for a symbol walks from the innermost scope outward, so inner declarations correctly shadow outer ones. Redeclaration checks only inspect the current (innermost) scope to allow valid shadowing.

5.1 Symbol Fields

Each symbol stored in the table contains the following fields:

- name: the identifier string
- kind: one of VAR, CONST, or FUNC

- type: one of BOOL, INT, FLOAT, CHAR, STRING, or VOID
- value: the current value of the symbol, used for type checking and constant evaluation
- isInit: whether the variable has been assigned a value (0 = uninitialized)
- isUsed: whether the variable has ever been referenced in an expression
- declLine: the source line number where the symbol was declared
- params / numParams: for function symbols, the list of parameter symbols and their count

5.2 Internal Architecture

The implementation is layered for extensibility: Symbol -> SymbolList -> ScopeSymbolTable -> SymbolTable. Each ScopeSymbolTable holds a linked list of Symbols for one scope. The SymbolTable maintains a linked list of ScopeSymbolTables, with the head always being the innermost (current) scope.

5.3 Output Files

- SymbolTable.out: a plain-text log of every scope push, pop, and declaration event, indented by scope depth.
- SymbolTableVisualiser.out: a formatted table per scope showing all declared symbols with their name, kind, type, value, and declaration line.

6. Semantic Analysis

Semantic checks are performed during parsing, embedded in the Bison grammar actions. Any semantic error is written to SemanticAnalysis.out along with the line number where it occurred. The following checks are implemented:

1. Variable redeclaration: if a name is already declared in the current scope, an error is reported.
2. Type mismatch on assignment: the right-hand expression type must match the variable's declared type. int and float are mutually compatible; all other types must match exactly.
3. Use of uninitialized variable: if a variable is used in an expression before being assigned a value, an error is reported.
4. Assignment to a constant: assigning a new value to a const symbol is an error.
5. Use of undeclared symbol: referencing a name not found in any reachable scope is an error.
6. Non-boolean condition: using a non-boolean expression as the condition in if, while, or repeat-until is an error.
7. Division by zero: if the divisor in a / or % expression is a literal zero, it is caught at compile time.
8. Calling a non-function: using a variable or constant with function-call syntax is an error.

9. Wrong argument count: calling a function with a different number of arguments than declared parameters is an error.
10. Argument type mismatch: argument types must match the corresponding parameter types (with int/float flexibility).
11. Return type mismatch: the type of the expression in a return statement must match the function's declared return type.
12. Return outside function: using a return statement outside of a function body is an error.
13. Return in void function: using return with an expression inside a void function is an error.
14. Nested function definition: defining a function inside another function is not allowed.
15. Void function in expression: a void function call cannot appear where a value is expected.

7. Quadruples

Quadruples are the intermediate representation (IR) generated by the compiler. Each quadruple has the form:

`(operation, argument1, argument2, result)`

When a field is not applicable, it is written as N/A. Temporary variables are named t0, t1, t2, ... and are generated automatically as expressions are evaluated. The special variable tr holds the return value of the most recently called non-void function. The special variable sr holds the switch expression during a switch block.

Quadruple	Description
(=, val, N/A, var)	Assigns val to var. Used for declarations with initializers and assignment statements.
(+, x, y, t)	t = x + y. Generated for addition expressions.
(-, x, y, t)	t = x - y. Generated for subtraction expressions.
(-, x, N/A, t)	t = -x. Generated for unary negation.
(*, x, y, t)	t = x * y. Generated for multiplication.
(/, x, y, t)	t = x / y. Generated for division.
(%, x, y, t)	t = x % y. Generated for integer remainder.
(^, x, y, t)	t = x ^ y. Generated for exponentiation.
(&, x, y, t)	t = x & y. Logical AND of two boolean values.
(|, x, y, t)	t = x | y. Logical OR of two boolean values.
(!, x, N/A, t)	t = !x. Logical NOT of a boolean value.
(==, x, y, t)	t = (x == y). Equality comparison; result t is boolean.
(!=, x, y, t)	t = (x != y). Inequality comparison; result t is boolean.

(<, x, y, t)	t = (x < y). Less-than comparison.
(>, x, y, t)	t = (x > y). Greater-than comparison.
(<=, x, y, t)	t = (x <= y). Less-than-or-equal comparison.
(>=, x, y, t)	t = (x >= y). Greater-than-or-equal comparison.
(JMP, label, N/A, N/A)	Unconditional jump to label.
(JZ, label, N/A, N/A)	Jump to label if the last computed value is zero (false). Used for if, while, for, and switch-case conditions.
LABEL0:	A jump target. Labels are generated sequentially: LABEL0, LABEL1, etc.
PROC funcName	Marks the beginning of a function definition.
(POP, N/A, N/A, param)	Pops the top of the call stack into the parameter variable. One POP is emitted per parameter, in reverse order.
(RET, N/A, N/A, N/A)	Returns from the current function.
(PUSH, x, N/A, N/A)	Pushes argument x onto the call stack before a function call.
(CALL, funcName, N/A, N/A)	Calls function funcName. All arguments must be PUSHed first.
(=, expr, N/A, tr)	Stores the return value of a function into the special variable tr (temp return).
(PRINT, x, N/A, N/A)	Outputs the value of x.
(PUSH, sr, N/A, N/A)	Saves the switch register sr before entering a switch block.
(=, expr, N/A, sr)	Loads the switch expression into the special variable sr.
(POP, N/A, N/A, sr)	Restores sr after exiting a switch block.

8. How to Build and Run

8.1 Prerequisites

- flex (Lex implementation)
- bison (Yacc implementation)
- gcc
- make

8.2 Build

Run the following command in the project directory:

```
make
```

This triggers Bison on Parser.y, Flex on Lexer.l, compiles all .c files, and links them into Compiler.exe.

8.3 Run

```
./Compiler.exe src.txt
```

Where src.txt is a source file written in the language.

8.4 Output Files

- Quadruples.out: the intermediate representation of the compiled program.
- SymbolTable.out: a log of symbol table construction and destruction events.
- SymbolTableVisualiser.out: a formatted per-scope view of all declared symbols.
- SemanticAnalysis.out: any semantic errors found, with line numbers.
- Syntax errors are printed to stderr with the line number where the error occurred.